ARTICLE

A note on relative neighborhood graphs

JERZY WL JAROMCZYK, University of Kentucky, Lexington, KY, United States

MIROSŁAW KOWALUK, University of Warsaw, Institute of Informatics, Warsaw, MA, Poland

Published: 01 October 1987

Citation in BibTeX format

CG87: Third Annual symposium on Computational Geometry
June 8 - 10, 1987
Ontario, Waterloo, Canada

Conference Sponsors:
SIGACT
SIGGRAPH

A Note on Relative Neighborhood Graphs

Jerzy W. Jaromczyk [1] and *Miroslaw Kowaluk* [2]

1) University of Kentucky, Dep. of Comp. Sci.
Lexington, KY 40506-0027, USA
and
Warsaw University, Inst. of Informatics
00-901 Warsaw, PKiN 8 p., Poland

2) Warsaw University, Inst. of Mathematics
00-901 Warsaw, PKiN 9 p., Poland

Abstract:

Two new algorithms finding relative neighborhood graph RNG(V) for a set V of n points are presented.
The first algorithm solves this problem for input points in (R^2,L_p) metric space in time $O(n\alpha(n,n))$ if the Delaunay triangulation DT(V) is given. This time performance is achieved due to attractive and natural application of FIND-UNION data structure to represent so-called elimination forest of edges in DT(V).
The second algorithm solves the relative neighborhood graph problem in (R^d,L_p), $1<p<\infty$, metric space in time $O(n^2)$ when no three points in V form an isosceles triangle. The complexity analysis of this algorithm is based on some general facts pertaining to properties of equilateral triangles in the metric space (R^d,L_p).

1.Introduction

The present paper addresses two problems pertaining to Relative Neighborhood Graphs.
This attractive notion having several applications in pattern recognition has been studied by Toussaint [To] and other authors (see e.g. [S]).

Relative neighborhood graph for a set of points V in a metric space M with a metric *dist* is a graph G(V,E) such that there is an edge between points p and q if and only if

$$dist(p,q)\leq\max(dist(p,z),dist(q,z):z\in V-\{p,q\})$$

Relative neighborhood graph for V will be denoted by RNG(V) and the problem of finding relative neighborhood graph will be referred as RNG problem.

In this paper two following new results will be presented.

Theorem 1: The RNG(V) for a set V of n points in R^2 with the norm L_p can be found in $O(n\alpha(n,n))$ time and $O(n)$ space provided that the Delaunay triangulation DT(V) is given.□

The best known algorithm for R^2 with the norm L_2 was presented by Supowit, see [S]. His algorithm takes time $O(n\log n)$ to construct RNG(V) starting from Delaunay Triangulation of V.

Second result presented in this paper deals with relative neighborhood graphs in the higher dimensions and can be stated as follows:

Theorem 2: The RNG(V) for a set of n points in $R^d, d \geq 2$ with the norm L_p , $1 < p < \infty$ can be found in $O(n^2)$ time and $O(n)$ space if no three input points form an isosceles triangle. □

An analogous result for $R^d, d \geq 2$ with metric L_2 is due to Supowit, see [S].
So our theorem is a generalization of Supowit's result to metric spaces with L_p norm.

Organization of the paper is the following.
Next section will present geometrical background. Then algorithm for relative neighborhood graph on a plane R^2 with metric L_p will be design. In Section 4 an algorithm for RNG problem in (R^d, L_p) will be given. A brief discussion will conclude the paper.

2. Relative neighborhood graphs in R^2

This section presents a new algorithm for finding relative neighborhood graph of a finite set of points in the metric space (R^2, L_p) , $1 < p < \infty$. Let us recall that L_p is defined as following norm

$$d_p(q_1, q_2) = \left(\sum_{j=1}^{k} |x_j(q_1) - x_j(q_2)|^p \right)^{\frac{1}{p}}$$

Now let $V \subset R^2$ and $v, w \in V$.

$lune(v, w)$ is an intersection of two discs of radius $d_p(v, w)$ centered at points v and w respectively.

Thus RNG(V) can be defined equivalently as follows:

an edge $\overline{v\,w}$ exists iff $lune(v, w)$ contains no points of V in its interior.

If $z \in lune(v, w)$ then we say that z eliminates edge $\overline{v\,w}$.

Few notations more. By $S_p(v, r)$ we mean a sphere centered at v with radius r in the norm L_p. The boundary of the sphere will be called *circle*. For circles in R^2 a counterclockwise orientation is assumed.

The following lemma explains a relationship between RNG(V) and other geometrical structure on V, specifically Delaunay Triangulation DT(V) and Minimal Spanning Tree MST(V).

(A comprehensive presentation of Delaunay Triangulation as well as the other structures can be found in [E].)

Lemma 1.1: If V is a finite set of points in (R^2, L_p), $1 < p < \infty$, then $MST(V) \subseteq RNG(V) \subseteq DT(V)$. □

Lemma 1.1 is a natural generalization of the similar theorem for (R^2, L_2) presented in [To]. The proof for (R^d, L_p) case is not difficult and follows closely the ideas of the proof given in [To] and therefore can be omitted.
It is known that the Delaunay Triangulation for a set of n points in (R^2, L_p), $1 \leq p \leq \infty$, can be constructed in $O(n \log n)$ time (see [L], [I,W]). On the other hand the lower bound for RNG problem is $\Omega(n \log n)$ (see [S]) hence no better than $O(n \log n)$ algorithm for this problem can be expected.
Therefore the natural way to tackle the RNG problem is to split the process into two phases as suggested by Lemma 1.1:

- construct DT(V)
- eliminate edges from DT(V)-RNG(V)

The above approach was used in [S] to get $O(n \log n)$ algorithm for (R^2, L_2) . Note that the elimination phase of Supowit's algorithm runs in $O(n \log n)$ time.

The objective of this section is to design an algorithm that given DT(V) constructs RNG(V) in $o(n \log n)$ time.
We will achieve this goal on a basis of some nice geometrical properties of DT(V) edges not belonging to RNG(V). These properties give us a ground to avoid sweeping along sorted coordinates as opposed to Supowit's algorithm. Instead a FIND-UNION structure is employed giving the claimed performance of the algorithm.
Result shows once again a power of Delaunay Triangulation.

2.1 Geometrical properties of RNG(V) in R^2

This section begins with some observations pertaining to the geometry of relative neighborhood graphs in (R^2, L_p). Then the algorithm, a proof of its correctness and its complexity analysis will be given.

Let Δabc be a triangle. $lune(a,b)$ is divided by a line passing through a,b into two symmetric parts. Let $lune^+(a,b)$ stands for the part in the halfplane containing the point c, $lune^-(a,b)$ stands for another one. Assume that S_p^o is the smallest (circumscribing) L_p sphere containing the points a,b,c. The following simple geometrical fact holds.

Fact 2.1.: If $d_p(b,c) \leq d_p(a,c)$ then $lune^-(a,b) \cap lune(b,c) \subset S_p^o$.
Moreover if $\overline{ab}$ is the shortest edge in Δabc then $lune^+(a,b) \subset S_p^o$.

Proof: (1) Consider a circle C_p of the sphere $S_p(c, d_p(b,c))$. This circle intersects S_p^o entering it at the point b and leaving at a point z lying in the arc $c\hat{a}$ $(d_p(a,c) \leq d_p(b,c))$ (see figure 1). Therefore the arc $b\hat{z}$ of the circle C_p is totally contained in S_p^o and in consequence also $lune^-(a,b) \cap lune(b,c) \subset S_p^o$.

(2) Let $S_p^a = S_p(a, d_p(a,b))$ and $S_p^b = S_p(b, d_p(a,b))$. Recall that $\overline{ab}$ is the shortest edge. Hence circles of S_p^a and S_p^b intersect the circle of S_p^o in the arcs $b\hat{c}$ and $c\hat{a}$ respectively (see figure 2). It means that the circles of S_p^a and S_p^b intersect inside S_p^o. Because two circles in L_p can intersect in at most two points we have $lune^+(a,b) \subset S_p^o$.
This ends the proof. □

Three lemmas presented below give us a background to design the main algorithm.

Lemma 2.2: If Δabc is a triangle of DT(V) and $v \in V$ then v eliminates at most two edges of Δabc.

Proof: Assume that there is a point $v \in V$ that eliminates a,b,c. Then $v \in lune(a,b) \cap lune(a,c) \cap lune(b,c)$ and by virtue of the fact 1 $v \in S_p^o$ where S_p^o is the smallest sphere containing a,b,c. It leads to the contradiction because $\Delta abc \subset DT(V)$ and $v \in V$. □

Let $v \in V$ and Δabc be a triangle in DT(V).
Assume that v eliminates $\overline{ab}$. We say that v is external to Δabc with respect to $\overline{ab}$ if and only if c and v are separated by the line passing through $\overline{ab}$. By symmetry Δabc will be called external to v w.r.t. $\overline{ab}$.

Lemma 2.3.: Let $v,w \in V$ eliminate an edge $\overline{ab}$ of $\Delta abc \subset DT(V)$ and v,w be external to Δabc w.r.t. $\overline{ab}$. Then $\overline{ac}$ and $\overline{bc}$ are not both eliminated by v,w.

Proof: Assume that $v,w \in V$ eliminate $\overline{ab}$ and are external to Δabc w.r.t. $\overline{ab}$. It means that $v,w \in lune(a,b)$. Let $d_p(b,c) \leq d_p(a,c)$. By virtue of the fact 2.1 $lune(a,b) \cap lune(b,c) \subset S^o$ where S^o is the smallest sphere containing the points a,b,c. It implies that $w,v \notin lune(b,c)$ because $\Delta abc \subset DT(V)$. Hence v and w cannot eliminate $\overline{bc}$ what ends the proof. □

Lemma 2.3 states that two points eliminating a common edge of an external triangle can eliminate at most one of the remaining edges ot this triangle.

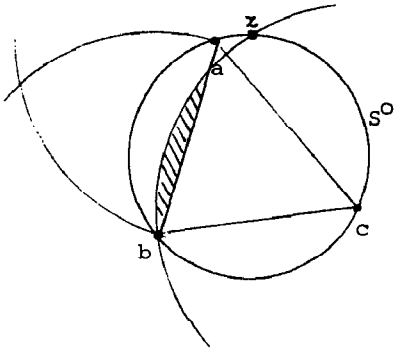


figure 1.

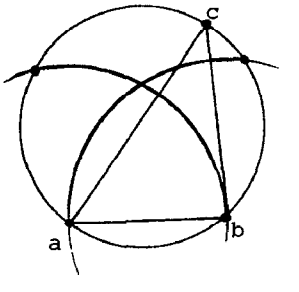


figure 2.

Now we are prepared to introduce two geometrical structures crucial for our algorithm : elimination path and elimination forest.

The elimination path for a point (starting from an adjacent triangle) is an ordered list of edges eliminated by this point. The elimination path is defined by the following construction (see algorithm *Elimination path*).

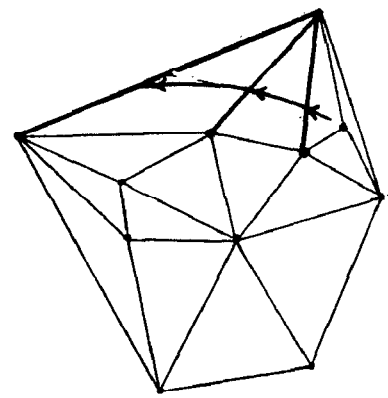
Observe that by the construction each pair of the adjacent edges in path $(\overline{bc},v)$ belong to the same triangle. By virtue of lemma 2.2 no three points in this path belong to the same triangle.

The above procedure examines edge of a triangle coming from certain direction (depending on value of new_triangle). Therefore a direction can be associated with each edge of elimination path. The direction depends on which triangle is an external to the point w.r.t. to the current edge in the elimination process .

Last edge in elimination path will be called root.

An example of an elimination path is given in figure 3.

The elimination path can be viewed as a graph with nodes (edges of DT(V) with the direction associated) connected by an oriented arrow representing the order in which the path is constructed. This interpretation will be used often hereafter. A subgraph of the elimination path is called a partial elimination path.

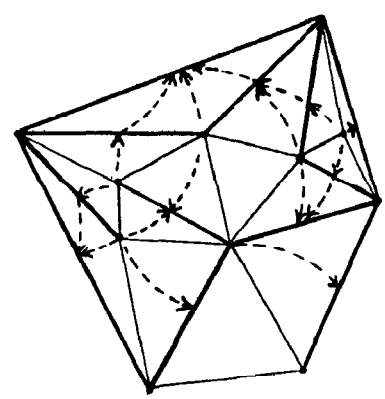


Bold edges are in elimination path

figure 3.

```
{input: edge bc ∈DT(V), point v ∈V}
begin
  path := ∅;
  if a eliminates b c
       then
        begin
          edge1 := b c;
          can_continue := true
        end
       else
         can_continue := false;
    while can_continue do
       begin
         path := append ( path, edge1 )
         new_triangle := triangle external to v w.r.t. edge1;
         if v eliminates a second edge edge2 of new_triangle
           then edge1 := edge2
           else can_continue := false
       end;
    path (bc,v) := path
end path;
```

Algorithm Elimination Path



Bold edges are in elimination forest

figure 4.

The next lemma explains a relation between elimination paths for different triangles and points.

Lemma 2.4.: Assume that two elimination paths have a common element (edge with the direction associated). Then starting from this element one of the paths is completely included in another one.
Proof: It is an immediate consequence of lemma 2.3. □

Lemma 2.4. implies that if the graph interpretation for elimination paths is used then a union of elimination paths is a forest of rooted and oriented trees. The union of all elimination paths will be called elimination forest. Any subset of the elimination forest will be called a partial elimination forest. An example of the elimination forest is shown on figure 4.
Observe that every edge of DT(V) can appear twice in the elimination forest - once in each direction of the elimination.

At this point a definition of the distance of a point to a segment is needed.
Let c be a point and $\overline{ab}$ be a segment. This distance is defined as $\overline{d}_p = inf\{d_p(c,x): x \in \overline{ab}\}$.
We say that a segment $\overline{de}$ lies between c and ab if $\overline{de} \cap \Delta abc \neq \emptyset$.

Now the crucial lemma follows.

Lemma 2.5: Let $\overline{ab} \in$ DT(V) and v be a closest (w.r.t. $\overline{d}_p$) point of V eliminating $\overline{ab}$. Then v eliminates all edges of DT(V) that lie between $\overline{ab}$ and v.
Proof: Assume that $\overline{zw} \in$DT(V) lie between v and $\overline{ab}$. Observe that $\overline{ab}$ and $\overline{zw}$ do not intersect (both belong to DT(V)). Also no endpoint of $\overline{zw}$ is inside Δabv because v is the closest to $\overline{ab}$ point in V (note that it holds for $\overline{d}_p$). Hence $\overline{zw}$ intersect both $\overline{av}$ and $\overline{bv}$.
Now some cases can be considered. Assume for example that $z, w \in lune(a,b)$. (see figure 5) Let $v \in \overline{ab}$ be such that $d_p(v,v') = \overline{d}_p(v,\overline{ab})$. Take a sphere $S_p(v', d_p(v,v'))$. This sphere intersects $\overline{zw}$ because $d_p(z,v'), d_p(w,v') \geq d_p(v,v')$. Now the circles of $S_p(w, d_p(v,w))$ and $S_p(v', d_p(v,v'))$ intersect in two points and convexity implies that $z \notin S_p(w, d_p(v,w))$. Hence $d_p(v,w) < d_p(z,w)$. The same reasoning shows that $d_p(v,z) < d_p(z,w)$. Therefore v eliminates $\overline{zw}$.
Another case, i.e., at most one of z, w belongs to $lune(a,b)$ is handled analogously. □

The above lemma implies directly

Corollary 2.6: Every edge in DT(V) - RNG(V) belongs to a certain elimination path, specifically to the one induced by the closest (w.r.t. $\overline{d}_p$) point in V.
Proof: Let $v \in$V be closest (w.r.t. $\overline{d}_p$) point to an edge e. By virtue of lemma 2.5 all the edges between v and e belong to the elimination path induced by v. Therefore e is also appended to this elimination path at a certain point (see algorithm *elimination path*). □
Now we are ready for the algorithm.

2.2 Algorithm for RNG problem in R^2

The idea standing behind the algorithm is the following. The algorithm starts from Delaunay triangulation DT(V) of the set V. Then the whole process is devoted to construct the elimination forest. It is carried out sequently by building elimination paths.
When an edge which is already in a certain tree $\mathbf{T}$ of the partial elimination forest is encountered in the process of building elimination path then "a jump" to the root of $\mathbf{T}$ can be performed (see lemma 2.4.) and the elimination path procedure is continued from this root (actually edge and a direction).
The jumps are essential for the sake of efficiency. One can easily give an example of a set that the total length of all its elimination paths is of the order $\Theta(n^2)$ (the length of each path is counted separately). Such the event is possible if there are many long overlays of elimination paths.
In order to obtain a desired performance of the algorithm a suitable representation for the objects is needed. To represent and to maintain the partial elimination forest the structure proposed in [GF] with path compression and union by rank is chosen (see [Ta2]).
More precisely, the algorithm will maintain the collection of disjoint sets $T_1,...,T_k$ consisting of the elements of the partial elimination forest $\mathbf{T}_1,...,\mathbf{T}_k$. It means that each T_i consists of pairs of the form <edge, direction> where direction is determined by the process of building the corresponding elimination path. The unique root of $\mathbf{T}_i$ serves as a representative of T_i.

Three operations to maintain a collection of T_i sets will be used (see e.g. [Ta2]):

makeset(e) - create a new set corresponding to {e}, e in no other set

find(e) - returns the representative of the set containing e

link(e,f) - forms a new set that is a union of two sets with representatives e and f and returns e as a representative of the new set ($e \neq f$)

The algorithm uses the array "Eliminate" of the edges of DT(V) (with two possible directions of the elimination) which initially are unmarked. A boolean variable "empty" is true if no element was put into the currently constructed elimination path.
The algorithm has the structure presented in Algorithm 1. We assume that DT(V) is given in the standard representation, see [E].

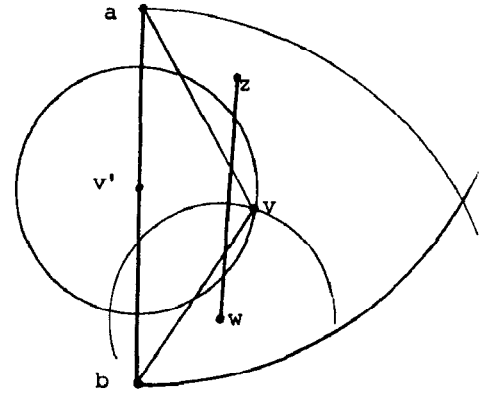


figure 5.

```
    Algorithm RNG(V)
{ input: Delaunay triangulation DT(V)}
{output: Relative neighborhood graph RNG(V)}
 for every ab ∈DT(V) and c such that Δabc⊂ DT(V) do
 begin
  empty :=true;
     { start constructing elimination path for ab,c }
     while a new edge e is eliminated by c do
      if Eliminate(<e,direction>) = marked
       then
         begin
            tree_root := find(<e, direction>);
            if not empty then link(tree_root, epath_repr);
            epath_rep := tree_root;
            empty :=false;
            continue constructing path(tree_root,c);
         end
       else
         begin
           Eliminate(<e,direction>) := marked;
           makeset(<e, direction>);
           empty := false;
           link (<e, direction>,epath_repr);
           epath_repr := <e, direction>;
           continue constructing path(e,c);
           end
   end;
  RNG(V) := DT(V) - {e : Eliminate(<e,direction>)= marked}
end of RNG ;
```

Algorithm 1 (RNG(V) for (R^2, L_p))

We will show that Algorithm 1 is correct.
Observe that every edge belonging to a certain elimination path is included in one of the sets constructed by the algorithm (and therefore is marked).
To see that assume that $T_1,...,T_k$ corresponding to the trees $\mathbf{T_1},...,\mathbf{T_k}$ of the partial elimination forest have been constructed and an edge e in the direction *dir* is to be appended to the path(e,c). If this edge is marked in the given direction it means that path(e,c) has a common element with another elimination path which elements are contained in, say $\mathbf{T_i}$ which is represented by T_i. Let the elements of the already constructed part of path(e,c) are contained in $\mathbf{T_j}$ represented by T_j. Now by virtue of lemma 2.4 either all elements of path(e,c) are in $\mathbf{T_i}$ or this path is continued from the root of $\mathbf{T_i}$ being a representative of T_i. So the link operation on T_i, T_j is justified and the search for the elements of the elimination path is carried out correctly. It is clear that the elimination forest contains only edges of DT(V) - RNG(V). Hence by virtue of corollary 2.6 the algorithm 1 is correct.

The overall cost of the algorithm depends on the cost to construct the disjoint sets representing the elimination forest. This construction is carried out by a sequence of find, link and makeset operations. The number of links is clearly not greater than the total number of the edges in the elimination forest, i.e., is not greater than $2 \times size(DT(V)) = O(n)$. Observe that the number of find operations is less than the total number of link operations. Recalling that the set union algorithm with path compression and union by rank runs in $O(n \times \alpha(n,n))$ for $O(n)$ link and find operations, where α is a functional inverse of Ackerman's function (see [Ta1]), we can conclude this section with the following

Theorem 1: The RNG(V) for a set V of n points in (R^2, L_p) can be found in $O(n\alpha(n,n))$ time and $O(n)$ space provided that the Delaunay triangulation DT(V) is given. □

3. Relative neighborhood graph for R^d

All considerations in this section refer to the space (R^d, L_p), where $d \geq 2$ and $1 \leq p \leq \infty$.
We will show a new algorithm for finding the relative neighborhood graph in these spaces.

Algorithm consists of two phases:

phase1 - constructing a supergraph of RNG(V)

phase2 - elimination those edges which do not belong to RNG(V)

Observe that this structure is similar to one used for the algorithm 1. The only difference is that the first phase here does not construct Delaunay Triangulation DT(V).
We will show that this supergraph (denoted by $G_o(V,E)$) can be constructed in $O(n^2)$ time and $card(E) = O(n)$.
This fact resulting from a geometry of equilateral triangles in (R^2, L_p) space allows to carry out the second phase of algorithm in $O(n^2)$ time.

The algorithm is presented in figure Algorithm 2.

```
Algorithm RNG in (R^d, L_p)
{Input: set V of n points in (R^d, L_p), d≥2, 1<p<∞,
        no three points of V form an isosceles triangle}
{output: Relative neighborhood graph RNG for V }

   begin
1.   E:=∅;
phase1: for each v_i ∈ V do
        begin
2.        E_i:= set of edges adjacent to v_i;
3.        while E_i ≠∅ do
                begin
4.                v_i v_j := minimal_length edge in E_i;
5.                E_i := E_i - {v_i v_j};
6.                E :=E ∪ {v_i v_k}
7.                for each v_i v_k ∈ E_i do
8.                  if d_p (v_i,v_k) > d_p (v_j,v_k)
9.                    then E_i :=E_i - {v_i v_k}
                end
        end;
phase2: for each edge v_i v_j ∈ E do
10.       for each point v ∈ V - {v_i ,v_j} do
11.         if v ∈ lune(v_i,v_j) then E :=E - {v_i v_j};
12.  RNG := E;
   end algorithm;
```

Algorithm 2 (RNG(V) for (R^d, L_p))

We will show that Algorithm 2 is correct.
Consider the first phase of the algorithm and the loop starting at the line 3. An edge $\overline{v_i v_k}$ is removed from E_i (in line 9) if $d_p(v_j,v_k) < d_p(v_i,v_k)$. Because $\overline{v_i v_j}$ is the shortest edge in E_i we have $d_p(v_i,v_j) < d_p(v_i,v_k)$ and in consequence $\overline{v_i v_j} \notin$ RNG(V). (There are no isosceles triangles so strict inequalities can be used).
After the first phase is completed E consists of all edges which were removed in the line 9 of the algorithm. Hence RNG(V) $\subset$ E.
The second phase examines directly each edge of E and each point of V removing only those edges which do not belong to RNG(V). Therefore Algorithm 2 solves RNG problem for V $\subset R^d$ in the metric space (R^d, L_p).
The complexity analysis of the Algorithm 2 is based on some general facts pertaining to the geometry of equilateral triangles in the metric space (R^d, L_p).

Fact 3.1. For every $d \geq 2$ there exists a constant $\pi(d) > 0$ such that each angle of any equilateral triangle in (R^d, L_p), $1 < p < \infty$, is greater than $\pi(d)$. □

The proof of the above fact is based on simple calculations and is omitted here. In particular one can show that $\pi(d) \geq \sin^{-1}(1/(2k))$

Fact 3.2. Let T be a triangle in (R^d, L_p). The angle opposite the longest edge of T is greater than $\pi(d)$.

Proof: Let P be a 2 dimensional plane passing through a, b, c. Consider this part W of the $lune(a,b) \cap P$ which contains the point c (see figure 6). Let C be the smallest (circumscribing) circle (in L_2) passing through a, b and containing W. Denote by z the point $C \cap W$ (different to a, b). Observe that such a point always exists. Now let z' be an intersection point of the boundary of $lune(a,b)$ and a L_p - sphere centered at z with a radius $d_p(a,b)$. Clearly $\Delta az'z$ is an equilateral triangle in L_p. Moreover $\overline{zz'}$ lies inside this sphere thus $\sphericalangle(azz') \leq \sphericalangle(azb)$ (convexity of L_p sphere). For all points inside the circle C, in particular for the point c, we have $\sphericalangle(acb) \geq \sphericalangle(azb)$ (relation between the angles in 2-dimensional L_2 circle). By virtue of the fact 3.1 we have $\sphericalangle(azz') \geq \pi(d)$, what ends the proof. □

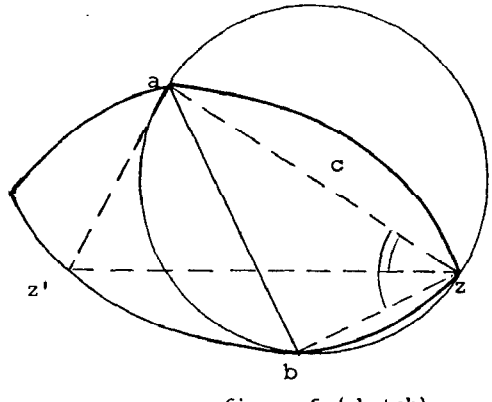


figure 6. (sketch)

Let E be the set of edges obtained after the first phase of the algorithm 2 is accomplished. The following fact holds.

Fact 3.3 Let $\overline{v_i v_j}$, $\overline{v_i v_k}$ be any edges in E adjacent to a vertex v_i. Then the angle $\sphericalangle(v_j, v_i, v_k)$ is greater than $\pi(d)$.

Proof: Let $e_j = \overline{v_i v_j}$, $e_k = \overline{v_i v_k}$ and assume that $d_p(v_i, v_j) < d_p(v_i, v_k)$. Accordingly to the algorithm 2 we have $d_p(v_j, v_k) > d_p(v_i, v_k)$ - otherwise $\overline{v_i v_k}$ would be removed in the line 9. Hence $\overline{v_k v_j}$ is the longest edge of the triangle Δabc and by virtue of fact 3.2 $\sphericalangle(v_j v_i v_k) > \pi(d)$. □

Note that absence of the isosceles triangles is essential in the above proof (strict inequalities are used).

Recall that G_o(V,E) stands for the graph constructed at the first phase of the algorithm.
Now the crucial lemma follows.

Lemma 3.4. There is a constant $c(d)$ such that the degree of any vertex v in G_o(V,E) is not greater than $c(d)$.

Proof: Let $v \in$ V. For every edge $e \in$ E adjacent to v we define a $cone(e,v)$ as follows.

$cone(e,v)$ is an open circular cone with the vertex v, symmetric w.r.t. the line passing through e and such that the angle at the vertex v is equal to $2/3\,\pi(d)$. Let $S_2(v,1)$ be a unit sphere in (R^d, L_2) centered at v. Now by the virtue of fact 3.3 $\text{cone}(e_1,v) \cap \text{cone}(e_2,v) = \emptyset$ for $e_1, e_2 \in$ E. Hence a sum of all volumes $\text{volume}(cone(e,v) \cap S_2(v,1))$ is smaller than the $\text{volume}(S_2(v,1))$. In turn the volume of each $cone(e,v) \cap S_2(v,1)$ depends on the angle $2/3\,\pi(d)$ only. Thus the number of the edges in E is bounded by a certain constant $c(d)$ depending on the dimension d only. □

The above lemma implies that (after the phase 1 of the algorithm) $card(\text{E}) = O(c(d) \times n)$. Therefore the cost of the loop starting at the line 3 is $O(c(d) \times n)$ and the overall cost of the phase 1 is $O(c(d) \times n^2)$. Cost of the phase 2 is proportional to $card(\text{E}) \times card(\text{V}) = O(c(d) \times n^2)$.

Remark also that the approach used above makes a cost analysis for the case of (R^d, L_2) fairly simple.

We can summarize the above discussion with the following theorem.

Theorem 2: The RNG(V) for a set of n points in $R^d, d \geq 2$ with the norm L_p, $1 < p < \infty$ can be found in $O(c(d) \times n^2)$ time and $O(n)$ space if no three input points form an isosceles triangle ($c(d)$ is a constant depending on the dimension d only). □

4. Remarks

In the paper two algorithms finding relative neighborhood graph RNG(V) for a set V of n points have been presented.

The first algorithm solves this problem for points in (R^2, L_p) metric space in time $O(n\,\alpha(n,n))$ if the Delaunay triangulation DT(V) is given. This time performance is achieved due to attractive and natural application of FIND-UNION data structure to represent so-called elimination forest of edges in DT(V).

It would be interesting to know whether RNG(V) can be found in $O(n)$ time provided that DT(V) is available.

The second algorithm solves the relative neighborhood graph problem in (R^d, L_p), $1 < p < \infty$, metric space in time $O(n^2)$ when no three points in V form an isosceles triangle.

(It is worth noting that this algorithm finds the graph of "relatively close" points (see [La]), defined like RNG except that the sharp inequalities are used.)

Again it would be interesting to have $O(n^2)$ algorithm not assuming the absence of isosceles triangles.

References:

[E] Edelsbrunner, H., Algorithms in Combinatorial Geometry, Springer Verlag, 1987.

[GF] Galler, B.A. and M.J. Fischer, "An Improved Equivalence Algorithm," Comm.ACM, vol. 7, pp. 301-303, 1964.

[La] Lankford, P.M., "Regionalization: theory and alternative algorithms," Geogr. Anal., vol. 1, pp. 196-212, 1969.

[LW] Lee, D.T. and C.K. Wong, "Voronoi Diagrams in $L_1(L_\infty)$ Metrics with 2-Dimensional Storage Applications," SIAM J.Comput., vol. 9, no. 1, pp. 200-211, February 1980.

[L] Lee, D.T., "Two-Dimensional Voronoi Diagrams in the L_p Metric," J.ACM, vol. 27, no. 4, pp. 604-618, October 1980.

[S] Supowit, K.J., "The Relative Neighborhood Graph, with an Application to Minimum Spanning Trees," J.Assoc.Comput.Mach., vol.30, no.3, pp.428-448, July 1983.

[T1] Tarjan, R.E., "Efficiency of a Good but not Linear Set Union Algorithm," J.Assoc.Comput.Mach., vol. 22, pp. 215-225, 1975.

[T2] Tarjan, R.E., Data Structures and Network Algorithms, Society for Industrial and Applied Mathematics, Philadelphia, Pennsylvania, 1983.

[To] Toussaint, G.T., "The Relative Neighborhood Graph of a Finite Planar Set," Pattern Recog., vol. 12, pp. 261-268, 1980.